基于螺旋路径的板凳龙运动模拟与优化模型构建

摘要

“板凳龙”是浙闽地区独特的传统民俗文化活动，数十乃至上百条板凳首尾相连，随着龙头的引领，龙身和龙尾随之盘旋，呈现出令人叹为观止的视觉效果。这背后蕴含着板凳龙独特的物理特性和运动规律。特别是在舞龙沿螺线盘入和盘出的过程中，如何确保板凳之间不发生碰撞、如何确定合适的螺距以保证顺利前进、以及如何设计合理的调头路径以实现从盘入到盘出的顺畅切换，都是亟待解决的问题。

针对问题一，构建**阿基米德螺线**运动模型，主要解决了“板凳龙”在运动中的位置变化，采用**时间延迟**与相对距离的原理，通过**迭代**得到不同时间点的位置；采用了改进的**牛顿-柯斯特公式**，**五次曲线逼近法**并采用**辛普森法则**计算验证位置的准确性。接着，运用弧长公式通过曲率分析与递归算法计算速度，相应结果详见 result1.xlsx 文件。

针对问题二，构建碰撞临界时间点模型，通过**旋转矩阵**计算板凳外顶点的位置，并以板凳间存在的几何关系，建立**板凳外顶点**到**理想化**其外圈板凳的距离函数，通过**数值求解**解出**非线性方程组**，计算出终止时刻，并计算出该时刻下舞龙队的位置和速度，相应结果详见 result2.xlsx 文件。

针对问题三，在问题一与问题二的基础上，研究板凳间碰撞的时间和到达掉头区域的时间两者与螺距的关系，并运用**模拟退火**方法，找出使得碰撞时间点最接近龙头到达掉头空间边界的时间点的螺距，即为最小螺距。

针对问题四，求解在调头圈内龙头前把手进行调头的最短路径，此问涉及一系列规划优化解答。同时，本模型采用极限的思想，通过数学几何的分析，结合极限思想的逼近求解，得到了本龙头前把手质点在调头中的可用极限时间。同时结合各种几何关系分析，运用了遗传算法进行模型圆弧运动的参数优化。

针对问题五，在问题四求出路径的基础上，根据路径曲率变化对运动把手速度和加速度的影响，得到在半径较小圆弧上进行加速达到速度最大，通过前面问题中的速度方程，求所有把手中出现的最大速度与龙头速度有关的方程，通过粒子群算法优化求出龙头允许的最大速度。

关键字：阿基米德螺旋线　递归算法　螺旋参数方程　时间延迟　数值积分法　极限

一、 问题重述

“板凳龙”是浙闽地区的传统地方习俗，“板凳龙”由几十条到上百条的板凳节节相连，龙头，龙身和龙尾相随盘旋形成螺线，这背后蕴含着其独特的物理特性和运动规律。舞龙队的表现需在有限的活动面积内完成，面积越小、速度越快，观赏效果就越出色，然而，如何确保板凳之间不发生碰撞、如何确定合适的螺距、以及如何设计合理的调头路径以实现从盘入到盘出的顺畅切换，是保障板凳龙表演安全与精彩的重要要素。

“板凳龙”结构“板凳龙”由 223 节板凳组成，其中第 1 节为龙头，接下来的 221 节为龙身，最后 1 节为龙尾。且只有龙头的板凳长度为 341 厘米，其它板凳长度均为 220 厘米，所有板凳的板宽均为 30 厘米。每节板凳上都有两个孔，孔的直径为 5.5 厘米，孔的中心距离板头最近的边缘为 27.5 厘米。相邻的两节板凳通过把手连接在一起。

问题 1 路径模拟和速度获取：现等距螺线的螺距为 55cm。龙头前把手速度恒为 1m/s。龙头从螺线第 16 圈的 A 点出发计时到 300s 为止，整个表演流畅情况下，求每秒整个舞龙队的位置和速度，以及计算出不同节点的位置和速度信息。**问题 2** 碰撞检测和终止时刻状态获取：在问题 1 的基础上，继续盘入螺线，确定舞龙队在各板凳恰好未碰撞时的终止时刻。当舞龙队继续盘入可能导致板凳之间（即各把手之间）发生碰撞时间即为终止时刻，本问题要求计算这一终止时刻，并计算此时舞龙队各把手的具体位置和速度。

问题 3 计算最小螺距：舞龙队沿螺线由顺时针进入调头空间，调头空间为以螺线中心作圆心、半径 4.5m 的圆形区域。求龙头前把手沿螺线盘入至调头空间边界时的最小螺距。

问题 4 调头路径优化：盘入螺线螺距为 1.7m，盘出螺线关于螺线中心呈中心对称。舞龙队在问题 3 设定的调头空间内完成调头，调头路径是由两段相切的圆弧组成的 S 形曲线，前段圆弧半径是后段的 2 倍，并且与盘入、盘出螺线相切。探讨在保持各部分相切的前提下，优化调头曲线以达到最小值。以调头开始时间为 0s，请问从 −100s 至 100s 每秒各板凳的位置和速度。同时列出几个时间节点时整队不同节点的位置和速度信息。

问题 5 求解龙头最大行进速度：延续问题 4 的路径设定，龙头行进速度恒定。要求舞龙队各把手速度均不超过 2 m/s，确定龙头的最大行进速度。

二、 问题分析

2.1 问题一分析

在问题一中，题目要求对舞龙队进行路径模拟和速度获取。首先，我们得到阿基米德螺线的螺线方程（即所有把手的运动路径），由已知龙头前进速度恒定不变，推出龙头每个时刻的位置，再使用时间延迟和相对位置来迭代推算出后面各点的位置。对于求出的位置，需要用数值积分方法进行验证。接着，使用螺线方程的弧长方程，曲率分析，使用递归算法进行速度的求解。

2.2 问题二分析

在问题二中，题目需要对碰撞检测和终止时刻状态获取进行建模。通过模拟仿真可以推出碰撞时的状态。接着分析几何关系，找出板凳顶点和外圈板凳碰撞的临界值的几何条件。构建外顶点到外圈相邻两点形成的线段的所在直线的距离函数，使其等于板宽的一半，对该非线性方程进行求解，从而求出终止时刻。最后，根据终止时刻求出题目要求的位置和速度。

2.3 问题三分析

在问题三中，题目需要在问题一与问题二的基础上，研究板凳间碰撞的时间和到达掉头区域的时间两者与螺距的关系，求出满足龙头前把手沿螺线盘入至调头空间边界时的最小螺距。通过分析得出我们需要保证到达圆形边界之前板凳龙不发生碰撞。最小螺距即为：碰撞时间点最接近龙头到达掉头空间边界的时间点的螺距。

2.4 问题四分析

在问题四中，要求解答调头圈内龙头前把手进行调头的最短路径，以及各时刻的运动轨迹分析。本模型分为两种情况进行讨论：情况 A 为现实中考虑曲率对螺线运动速度的影响因子以及距离偏差的运动情况；而情况 B 为理想化下的完全光滑且连续的圆周运动。对这两种情况进行比较式论证，逐渐得出：由参数差异引发的最短通行调头时间的差异。同时，这道题目需要考虑多种约束条件：如几何约束，速度约束，时间约束，空间约束等等多维度进行参数空间的限制。因此需要同时结合非线性的优化和规划思想，通过有限的参数空间 (非线性规划) 求解得到的局部最优解的调头路径。

2.5 问题五分析

在问题五中，舞龙队沿问题 4 设定的 S 形路径行进，要保证所有点在运动过程中速度最大的数值不能超过 2m/s。龙身和龙尾的速度和加速度会受到路径曲率的影响，随

着半径的增加，它们的速度会逐渐降低。舞龙队先进入半径较小圆弧，此时的半径小于掉头空间的半径，龙头线速度保持不变的情况下，舞龙队后面把手的速度会获得一个数值为正的加速度。再进入半径较大的圆弧时，舞龙队后面把手的速度会获得一个数值为负的加速度。因此速度最大值会出现在半径较小的圆弧上。可以使用粒子群算法优化得出 V_max。

三、 模型假设

为了模型更加合理简化问题，本文建立以下假设：

- 相邻板凳之间的连接是刚性的，即没有任何弯曲或变形，板凳的形状和尺寸始终保持不变。
- 理想化所有把手的正中心始终精准地在螺线上运动，不发生任何偏移，理想化为质点。
- 忽略重力风阻以及向心力的作用
- 盘龙的路径是一个二维平面上的螺线，忽略了舞龙队员由于身高不一样带来的影响。
- 忽略板凳连接处叠加的厚度的影响，忽略摩擦阻力对速度的影响。

四、 符号说明

符号	说明	单位
r	运动点（把手中心）到坐标系中心原点的距离长度	m
t	时间	s
R_0	运动中心轨迹最大极径	m
p	螺距	m
b	螺距系数，由周增长半径与完整螺旋圈数	m/π
$v_{i,t}$	第 i 个把手在 t 时刻的前进速度	m/s
θ	运动点（把手中心）在运动过程时刻 t 对应点与坐标 x 轴的夹角	$^\circ$
α	单个板凳的顶点连接最近的把手中心与板凳内前后把手连接得直线的锐角夹角	rad
ω	随时间变化的角速度	rad
$\mathrm{d}s$	代表沿运动轨迹劣弧线段的长度	m
D	龙头板凳靠外圈的顶点与该板凳所在螺线外圈的某两个前把手形成的直线的垂直距离	m
S	龙头板凳靠外圈的顶点与该板凳所在螺线外圈的某两个前把手形成的直线的垂直距离	m
D_i	第 i 个点距离龙头前把手的总距离	m
t_i	第 i 个点相对于龙头位置的相对延迟时间	s

五、 问题一：螺线运动模型的建立与求解

5.1 简本原理

5.1.1 阿基米德螺线

根据问题描述，这是一个涉及**阿基米德螺线**的特殊运动学问题，其中 A 点作为龙头的起始位置，其运动中心的轨迹最大极径为 R_0。根据已知条件 $p = 0.55\,\mathrm{m}$，计算 R 的公式如下：

$$R_0 = 16 \times p = 8.8\,\mathrm{m} \tag{1}$$

5.1.2 螺线运动模型建立

现已知运动中心的最大极径 $R_0 = 8.8\,\mathrm{m}$，并且螺距系数 $b = -\frac{p}{2\pi}$。结合阿基米德螺线的极坐标方程 $r(\theta) = a + b\theta$，可以将螺线的极径表示为：

$$r(\theta) = R_0 + b\theta = 8.8 - \frac{0.55}{2\pi}\theta \tag{2}$$

5.2 模型分析与求解

5.2.1 对极角的探究

5.2.1.1 龙头的运动轨迹

考虑到在螺旋运动过程中，极角 θ 随时间 t 的变化是非线性的，不同于匀速圆周运动。因此，在时间区间 $t \in [0, 300]$ 秒内，我们需要对每个时间间隔内运动点与原点连线扫过的角度增量 $\Delta\theta$ 进行精确的积分计算。

5.2.1.2 弧长微分分析

为了深入分析螺线的运动轨迹，我们可以考察螺旋线的一微小区间，通过弧长微分分析其轨迹特性。弧长微分公式为：

$$\mathrm{d}s = \sqrt{(R_0 + b\theta)^2 + \left(\frac{\mathrm{d}R_0}{\mathrm{d}\theta}\right)^2}\,\mathrm{d}\theta \tag{3}$$

其中，根据螺距系数的定义，极径对极角的导数为：

$$\frac{\mathrm{d}R_0}{\mathrm{d}\theta} = b = -\frac{0.55}{2\pi} \tag{4}$$

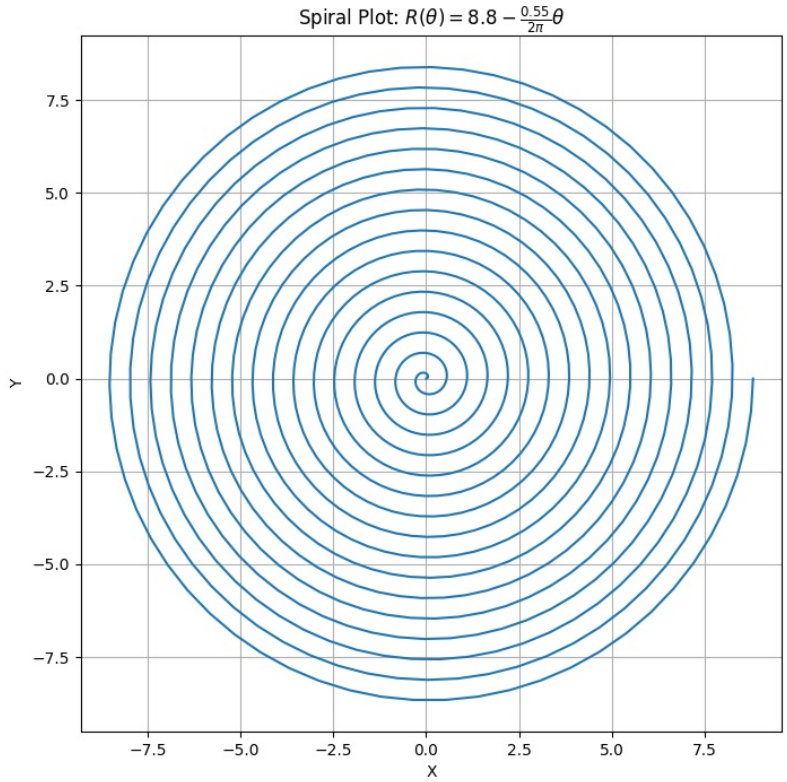


图 1　0.55m 螺距的螺旋线图

将上述导数代入弧长微分公式（3），我们可以更进一步得出如下表达式，考虑龙头前把手的行进速度为恒定的 1 m/s：

$$v = \frac{\mathrm{d}s}{\mathrm{d}t} = 1 = \frac{\mathrm{d}\theta}{\mathrm{d}t}\sqrt{(R_0 + b\theta)^2 + \left(\frac{\mathrm{d}R_0}{\mathrm{d}\theta}\right)^2} \tag{5}$$

因此，对于龙头的轨迹，我们有：

$$\frac{\mathrm{d}s}{\mathrm{d}t} = \sqrt{(R_0 + b\theta)^2 + b^2}\frac{\mathrm{d}\theta}{\mathrm{d}t} = 1 \tag{6}$$

计算从起始时间 t=0 至某个时间点 t 的时序区间 $[0, t]$。

为了求解时间 t 内扫过的极角 $\theta(t)$，我们对上述公式进行积分：

$$\int_0^t \mathrm{d}t = \int_0^{\theta(t)} \frac{1}{\sqrt{(R_0 + b\theta)^2 + b^2}}\,\mathrm{d}\theta \tag{7}$$

简化得：

$$t = \int_0^{\theta(t)} \frac{1}{\sqrt{(R_0 + b\theta)^2 + b^2}}\,\mathrm{d}\theta \tag{8}$$

换元表达：令 $u = R + b\theta$，则有 $\mathrm{d}u = b\,\mathrm{d}\theta$，从而 $\mathrm{d}\theta = \frac{\mathrm{d}u}{b}$。积分表达式变为：

$$t = \int_0^{\theta(t)} \frac{1}{\sqrt{(R_0 + b\theta)^2 + b^2}}\,\mathrm{d}\theta \tag{9}$$

换元后：

$$u(t) = R_0 + b\theta(t) \tag{10}$$

调整积分上限 $u(t) = R_0 + b\theta(t)$，调整积分下限 $u(0) = R_0$，积分表达式为：

$$\begin{aligned} t &= \int_{R_0}^{R_0+b\theta(t)} \frac{1}{\sqrt{u^2 + b^2}}\,\frac{\mathrm{d}u}{b} \\ &= \frac{1}{b}\int_{R_0}^{R_0+b\theta(t)} \frac{\mathrm{d}u}{\sqrt{u^2 + b^2}} \end{aligned} \tag{11}$$

求解结果代入得：

$$t = \frac{1}{b}\left(\frac{\ln(R_0 + b\theta(t)) + \sqrt{(R_0 + b\theta(t))^2 + b^2}}{R_0 + \sqrt{R_0^2 + b^2}}\right) \tag{12}$$

对数化操作及换元后，令 $c = R_0 + b\theta(t) + \sqrt{(R_0 + b\theta(t))^2 + b^2}$，则：

$$a + b\theta(t) = \frac{c^2 - b^2}{2c} \tag{13}$$

进一步简化：

$$\theta(t) = \frac{e^{bt}\left(R_0 + \sqrt{R_0^2 + b^2}\right) - \sqrt{e^{bt}\left(R_0 + \sqrt{R_0^2 + b^2}\right) - b^2 - 2a}}{2b} \tag{14}$$

由于公式较为复杂，本文将运用数值积分来解决该方程：在时间 t 维度上，对运动点的极角进行积分。

为了解决隐式表达形式的数值积分方法，本模型提出了一种对牛顿-柯斯特公式的改进，通过五次曲线逼近的方式来取代普通的梯形积分法则或矩形积分，从而进一步地逼近定积分中数值的近似解。

运用二次插值的思想，将被积函数 $\sqrt{R_0^2 + b^2}$ 在规定区间 $[t, t+1]$ 来逼近，逐次逼近在时间区间 $[0, 300]$ 内的近似积分。

由于原函数具有光滑的性质，所以为了提高建模精度，采用离散时间步长：时间离散为 n 个子步，步长为 $\Delta t = 1$。在问题 1 中，$n = 300$，时间离散点为 $t_0, t_1, t_2, \ldots, t_n$。其中 $t_0 = 0$, $t_n = n$

由辛普森法则得：

$$\theta(t) \approx \frac{\Delta t}{6}\left[f(t_0) + 4\sum_{i\text{奇数}} f(t_i) + 2\sum_{i\text{偶数}} f(t_i) + f(t_n)\right]$$
$$= \frac{f(t_0) + 4\sum_{i\text{奇数}} f(t_i) + 2\sum_{i\text{偶数}} f(t_i) + f(t_n)}{6} \tag{15}$$

其中：

$$f(t_i) = \frac{1}{\sqrt{(R_0 + b\theta_i)^2 + b^2}} \tag{16}$$

5.2.2 位置求解

5.2.2.1 龙头的角度和半径变化

对于龙头前把手，角度随时间 t 增加，并且有线性关系：

$$\theta_{\text{head}}(t) = \frac{v_{0,t}t}{R_0} \tag{17}$$

其中 v 是龙头的前进速度 (1 m/s)，龙头的半径则为：

$$r_{\text{head}}(t) = R_0 - \frac{p\theta_{\text{head}}(t)}{2\pi} = R_0 - \frac{pv_{0,t}t}{2\pi \cdot R_0} \tag{18}$$

5.2.2.2 第 i 个点的相对距离

第 i 个点的距离龙头的总距离 D_i，可以分成两部分。

- 龙头前把手到龙头后把手（即第一节龙身前把手）的距离：给定为 2.86 m
- 各相邻龙身前把手之间的距离：1.65 m

因此，第 i 个点（从龙头前把手开始编号，龙头前把手为第一个点）距离龙头前把手的总距离为：

$$D_i = 2.86 + (i-1) \cdot 1.65 \tag{19}$$

5.2.2.3 第 i 个点与龙头相比有时间延迟效应

在经典力学中，相对运动理论用于分析两个物体在不同位置、不同时间的运动关系。若龙头的相对时间位置为 t_{head}，则第 i 个点相对于龙头位置的相对延迟时间：

$$t_i = \frac{D_i}{v} \tag{20}$$

式中 v 是龙头的线速度 1 m/s。每个点的位置可以用角度 $\theta_i(t) = \theta_{\text{head}}(t - t_i)$ 进行表示。半径变化：

$$r_i(t) = R_0 + b\theta_i(t) \tag{21}$$

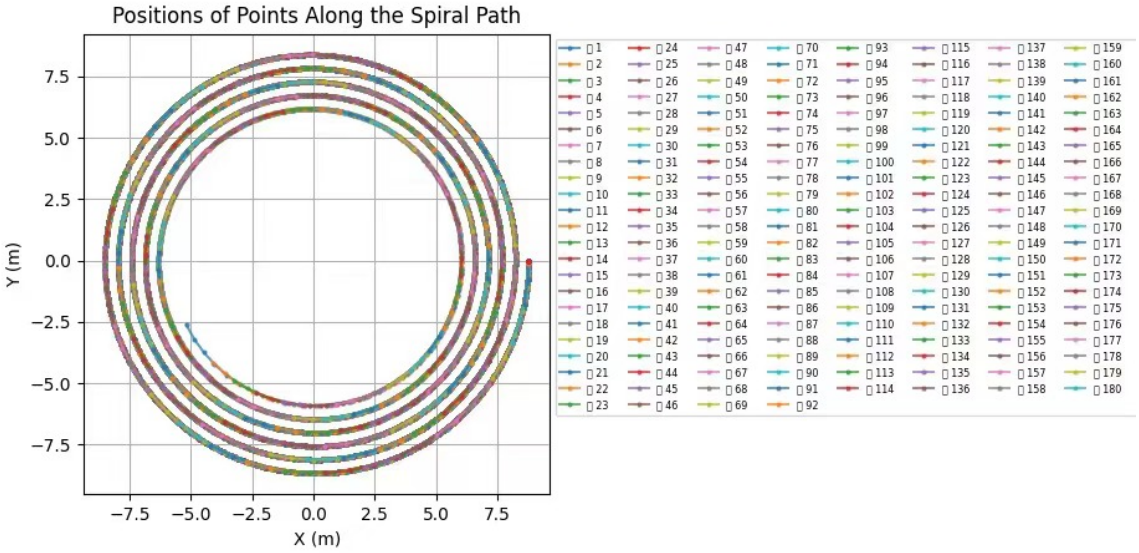


图 2　0s 至 300s 各把手运动位置显示图

5.2.3 位置验证

已知 $\theta_{\text{head}}(t) = \frac{v_{0,t}t}{R_0}$, 由阿基米德螺旋线极坐标方程 $r(\theta) = R_0 + b\theta$j，计算出龙头前把手点扫过的极角 θ 与时间 t 的关系函数。

结合上式可得：任意时刻下龙头前把手中心点的坐标位置 $(X_{1,t}, Y_{1,t})$：

$$X_{1,t} = r\,|\cos\theta \mod 180°|$$

$$Y_{1,t} = r\,|\sin\theta \mod 180°|$$

龙头前把手中心点的运动轨迹方程 $(X_{1,t}, Y_{1,t})$ 已求得，且同一块板凳上的两个把手中心点间距 l 是固定值：

$$l_{\text{龙头}} = \frac{341 - 27.5 \times 2}{100}\text{m}$$

$$l_{\text{龙身和龙尾}} = \frac{220 - 27.5 \times 2}{100}\text{m}$$

则可以通过：以龙头前把手中心点为圆心（设为点 H），板凳上两把手中心间距 $l_{龙头}$ 为半径 r_l 作圆与盘入轨迹交点。

观察可知，存在多个交点。

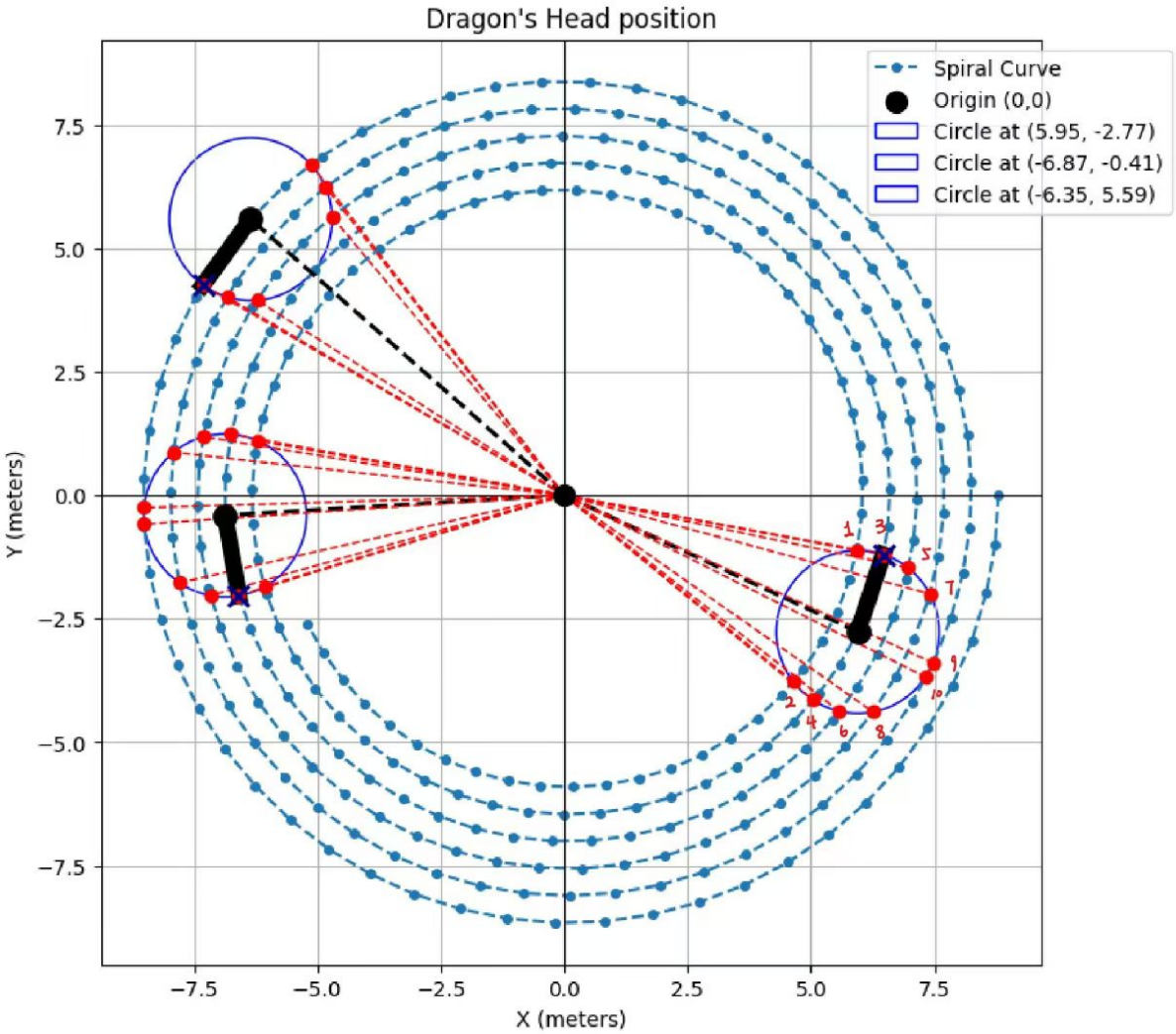


图 3　下一个把手中心点求证分布图

由于 $l_{龙头} = 286cm$、$l_{龙身和龙尾} = 165cm$、$p = 55cm$，易知当点 H 往内至少完成完整一圈后，以 H 为圆心、以 r_l 为半径的圆必定与盘入轨道有六个及以上的交点。

其中，

- 交点 1、2：H 当前盘入轨道的下一圈轨道
- 交点 3、4：H 当前盘入轨道的同一圈轨道
- 交点 5、6：H 当前盘入轨道的上一圈轨道
- 交点 7、8：H 当前盘入轨道的上两圈轨道
- 交点 9、10：H 当前盘入轨道的上三圈轨道

观察图 3，通过比较找出了三组中交点分别与原点的距离同 H 点与原点距离较接

表 1　选定圆（-6.87，0.41）的交点结果（转置）

交点	1	2	3	4	5	6	7	8	9	10
距离 (m)	8.53	8.51	7.99	7.96	7.44	7.40	6.90	6.85	6.34	6.30

近的交点 3、4。

进一步验证：由于 H 点始终在行列的前端且绕顺时针盘入，通过计算交点 3、4 各自极角值，取较小点作为紧接 H 的下一个相邻把手。

$$\min\left\{\left|\theta_3 \mod 180^\circ\right|, \left|\theta_4 \mod 180^\circ\right|\right\} \tag{22}$$

经验证：只有点 3(-6.59, -2.03) 成立。

龙身部分的把手运动同理与上述推断。

5.2.4 速度求解

由于龙头前把手盘入轨迹的速度 $v_{0,t} = 1\text{m/s}$ 保持恒定，且相邻把手的关联性及龙身与龙尾结构的一致性，因此运用递归算法，逐次调用往前一个把手的速度信息，直至龙头前把手的速度 $v_{0,t} = 1\text{m/s}$ 处达到递归的终止条件。

同时根据阿基米德螺线定律，若运动点的极角变化较大，则需要考虑“螺线曲率变化对运动速度的影响”。

根据弧长公式 $\sqrt{(R_0 + b\theta)^2 + b^2}\mathrm{d}\theta$ 可求得把手某一时间段扫过的极角之差对应的盘入轨迹弧长距离：

$$S = \int_{\theta 1}^{\theta 2} \sqrt{(R_0 + b\theta)^2 + b^2}\mathrm{d}\theta \tag{23}$$

检验：

研究各质点匀速运动位置在不同情况下的随时间变化的曲率分析

观察图 1，“板凳龙”在阿基米德螺线上盘入过程中，极角单位时间内变化较为剧烈，而图 2 中圆周运动的曲率变化为 0。故不能将单位时间内把手的螺线运动轨迹简化为圆周运动，需要考虑螺线中的曲率的实际变化。

现控制时间步长一致 $\Delta t = 1\text{s}$，则曲率变化反映在把手于盘入轨迹弧线运动的距离。根据关于 S 的式子（24），由于时间步长一致，则计算出关系：

$$\frac{S_{i+1}}{S_i} = \frac{v_{i+1}t}{v_i t}$$

$$= \frac{\int_{\theta_i}^{\theta_{i+1}} \sqrt{(R_0 + b\theta)^2 + b^2}\mathrm{d}\theta}{\int_{\theta_i}^{\theta_{i+1}} \sqrt{(R_0 + b\theta)^2 + b^2}\mathrm{d}\theta} \tag{24}$$

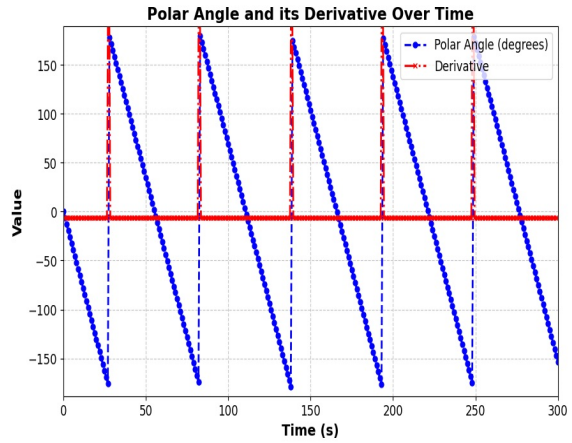


图 4　在阿基米德螺线上匀速盘入的极角变化值

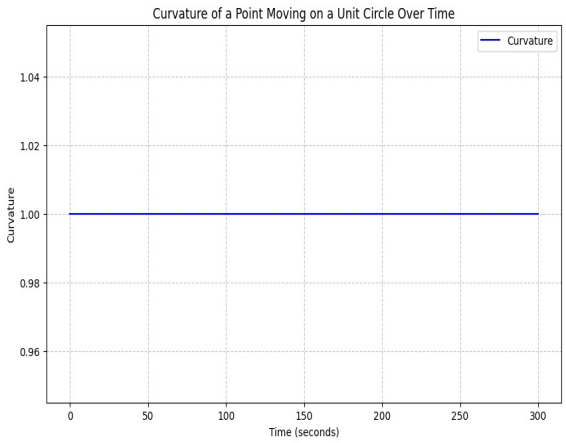


图 5　匀速运动质点在圆周运动下各位置的曲率

因此，

$$v_{i+1} \approx \frac{\sqrt{(R_0 + b\theta_{i+1})^2 + b^2}}{\sqrt{(R_0 + b\theta_i)^2 + b^2}} v_i \tag{25}$$

由之前的位置求解可知所有运动把手（$i \in [0, 224]$) 在各时刻 t($t \in [0, t_{end}]$) 的位置坐标点 $(X_{i,t}, Y_{i,t})$，通过反正切关系：位置坐标点的各个时刻的位置对应的夹角公式：

$$\theta_t = \text{atan2}(y_{i,t}, X_{i,t}) \times \frac{180}{\pi}, (\theta \in [-180°, 180°]) \tag{26}$$

综上得到速度计算的递归公式：

$$v_{i+1,t} = \sqrt{\frac{(R + b \times \text{atan2}(y_{i+1,t}, X_{i+1,t}))^2 + b^2}{(R + b \times \text{atan2}(y_{i,t}, X_{i,t}))^2 + b^2}}$$

$$v_{i,t} = \sqrt{\frac{(R + b \times \text{atan2}(y_{i,t}, X_{i,t}))^2 + b^2}{(R + b \times \text{atan2}(y_{i-1,t}, X_{i-1,t}))^2 + b^2}}$$

$$\vdots$$

$$v_{1,t} = \sqrt{\frac{(R + b \times \text{atan2}(y_{1,t}, X_{1,t}))^2 + b^2}{(R + b \times \text{atan2}(y_{0,t}, X_{0,t}))^2 + b^2}} \tag{27}$$

终止条件：$v_{0,t} = 1\text{m/s}$

根据上述递归表达式及终止条件设置递归函数代码，得到速度计算结果。

5.3 求解结果

本题位置与速度的总表储存于文件 result1.xlsx 中。

表 2　result1 中选取几个把手的位置

	0 s	60 s	120 s	180 s	240 s	300 s
龙头 x (m)	8.800000	7.056944	3.652034	-0.239631	-3.456110	-5.194122
龙头 y (m)	0.000000	-4.182288	-6.672258	-7.005410	-5.401640	-2.616324
第 1 节龙身 x (m)	8.800000	8.050783	5.612276	2.017960	-1.557272	-4.106789
第 1 节龙身 y (m)	0.000000	8.050783	-5.176131	-6.742452	-6.250039	-4.158131
第 51 节龙身 x (m)	8.800000	8.050783	5.612276	-1.875862	2.104111	4.913604
第 51 节龙身 y (m)	0.000000	0.000000	6.034667	7.631429	6.950251	4.503096
第 101 节龙身 x (m)	8.800000	8.800000	8.800000	1.649812	-2.723901	-5.763423
第 101 节龙身 y (m)	0.000000	0.000000	0.000000	-8.520995	-7.609581	-4.776699
第 151 节龙身 x (m)	8.800000	8.800000	8.800000	8.800000	8.800000	6.651077
第 151 节龙身 y (m)	0.000000	0.000000	0.000000	0.000000	0.000000	4.975590
第 201 节龙身 x (m)	8.800000	8.800000	8.800000	8.800000	8.800000	8.800000
第 201 节龙身 y (m)	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
龙尾（后）x (m)	8.800000	8.800000	8.800000	8.800000	8.800000	8.800000
龙尾（后）y (m)	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

表 3　result1 中选取几个把手的速度

	0 s	60 s	120 s	180 s	240 s	300 s
龙头 (m/s)	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000
第 1 节龙身 (m/s)	0.000000	0.986974	0.981696	0.976418	0.971141	0.965863
第 51 节龙身 (m/s)	0.000000	0.952681	0.957978	1.014348	1.008932	1.003515
第 101 节龙身 (m/s)	0.000000	0.952681	0.973666	1.052317	0.983219	0.977878
第 151 节龙身 (m/s)	0.000000	0.952681	0.973666	1.058565	0.954272	0.953127
第 201 节龙身 (m/s)	0.000000	0.952681	0.973666	1.058565	0.954272	0.975088
龙尾（后）(m/s)	0.000000	0.952681	0.973666	1.058565	0.954272	0.975088

六、 问题二：动态碰撞检测问题模型的建立与求解

6.1 模型建立

6.1.1 动态碰撞检测模型建立

考虑到两个板凳之间的衔接处会有 27.5cm 的伸长，内外圈两个把手之间的板凳区域不可能发生碰撞。碰撞只可能发生在板凳靠外圈的顶角与外圈的板凳之间。已知板

宽 w=30cm，所以碰撞临界值为：靠外圈的顶点与该板凳所在螺线外圈的某两个前把手形成的直线的垂直距离 D = w /2 ，该时刻为终止时刻。如图 6 所示。

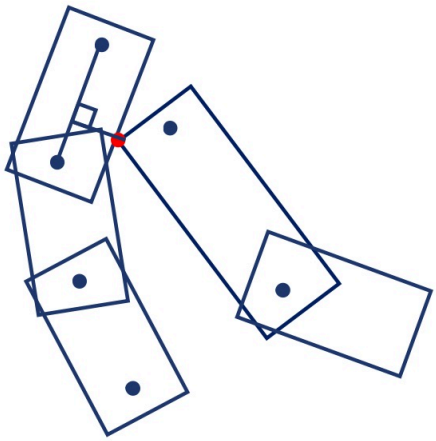


图 6　碰撞分析图

对于垂直距离 D，由点与两点形成的直线的垂直距离公式得出：

$$D = \frac{|(y_{i+1} - y_i) \cdot x_0 - (x_{i+1} - x_i) \cdot y_0 + x_{i+1} \cdot y_i - y_{i+1} \cdot x_i|}{\sqrt{(y_{i+1} - y_i)^2 + (x_{i+1} - x_i)^2}} \tag{28}$$

每个板凳顶点的位置可以通过其前把手的位置推断出，同时由于它会与外圈的板凳相碰撞：

旋转矩阵假设我们有一个固定点相对龙头的坐标 $(x_{\text{relative}}, y_{\text{relative}})$，并且我们需要将这个点的坐标从龙头坐标系转换到绝对坐标系。旋转矩阵用于将这个点从龙头坐标系旋转到绝对坐标系。

旋转矩阵的形式为：

$$R(\alpha) = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix}$$

其中，α 是旋转角度。旋转角度 α 满足 $\tan\alpha = \pm\frac{W}{2 \cdot l_{\text{把手与板凳头部}}}$

应用到固定点

1. **旋转矩阵**：

$$R(\alpha) = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix}$$

这里的 α 是从固定点相对位置的角度 $angle_a$ 中减去当前旋转角度 α，即 $angle_a - \alpha$。

2. **固定点相对坐标**：

$$\begin{bmatrix} x_{\text{relative}} \\ y_{\text{relative}} \end{bmatrix} = R(angle_a - \alpha) \begin{bmatrix} d \\ 0 \end{bmatrix}$$

计算得：

$$\begin{bmatrix} x_{\text{relative}} \\ y_{\text{relative}} \end{bmatrix} = \begin{bmatrix} d\cos(angle_a \pm \alpha) \\ d\sin(angle_a \pm \alpha) \end{bmatrix}$$

3. **固定点绝对坐标**：将旋转后的相对坐标添加到龙头的绝对坐标上：

$$\begin{bmatrix} x_{\text{point}} \\ y_{\text{point}} \end{bmatrix} = \begin{bmatrix} x_{\text{head}} \\ y_{\text{head}} \end{bmatrix} + \begin{bmatrix} x_{\text{relative}} \\ y_{\text{relative}} \end{bmatrix}$$

代入方程之后使用 **fsolve 函数**解决垂直距离方程。求出终止时间 t=432s。如图 7 可知每个板块的顶点路径也是等距螺线。

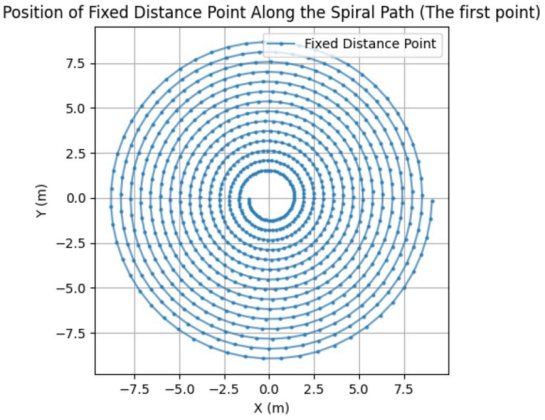


图 7　板凳外顶点运动轨迹

6.2 模型求解

根据题目一的坐标公式，可以得出所有的把手中心点的坐标。从龙头外顶点开始遍历，直到找到与附近板凳前后把手连线的垂直距离 =15cm 的时候停止。

6.3 求解结果

本题的位置与速度总表储存于文件 result2.xlsx 中。

七、 问题三：最小螺距优化问题分析

7.1 模型建立

调头空间是以螺线中心为圆心、直径为 9m 的圆形区域。等距螺线最后终点为原点，该螺线一定会与与掉头区域相交，能否进入调头空间与螺距大小五无关。由题目 2 推断

表 4　result2 中选取几个把手的位置和速度

	位置（x)	位置（y)	速度 (m/s)
龙头	0.670313959	0.670313959	1.000000
第 1 节龙身	0.558625207	0.768332724	1.001850
第 51 节龙身	-1.532555938	-1.901249917	0.970357
第 101 节龙身	2.618292856	2.93627216	1.000868
第 151 节龙身	-3.808366394	-3.865198688	0.969375
第 201 节龙身	5.094773965	4.680336379	0.999885
龙尾（后）	1.174163333	-7.483213738	0.978588

出，使得龙头前把手能够沿着相应的螺线盘入到调头空间的边界需要满足：当龙头到达圆形区域边界的时候，板凳之间恰好不发生碰撞，即板凳碰撞的时间点在到达掉头空间之后。

$$\begin{aligned} r(\theta) &= 8.8 - \frac{p}{2\pi}\theta \\ &= 4.5 \end{aligned}$$

7.2 求解结果

由问题 2 的公式得，我们希望找到的临界值为：碰撞时间点等于龙头恰好到达掉头空间边界的时间点。接下来，由于碰撞时间点等于龙头恰好到达掉头空间边界的时间点可能不存在，通过模拟退火，在有限误差之内，我们可以得出最小螺距 $p_{min} = 0.49m$。

八、 问题四：S 形掉头曲线及其长度优化问题分析

8.1 S 形掉头曲线问题建模思路

总体上划分两个思路进行讨论：

- 情况 A：实际螺线运动过程，综合考虑螺线曲率对速度的影响以及螺线实际运动轨迹等现实因素。
- 情况 B：理想化的圆周运动过程，即在龙头前把盘入及盘出过程中，运动路径严格遵循光滑、连续的圆周表面。B 情况下理想化掉头过程，不需要按照 S 型曲线轨迹，而是直接在可忽略不计的时间内完成运动速度方向的反转。

本模型引入了极限的思想：由于龙头前把手速度 $v_{0,t}$ 保持恒定，故第四题中分析圆弧调头路径的最小值规划问题，可转化为求解圆弧调头的可用极限时间 T 极限。

表 5　本题符号说明

符号	说明	单位
$R_{\text{调头}}$	调头区域的圆半径	m
M	进入调头阶段前一瞬间的位置标记点	-
N	离开调头阶段前一瞬间的位置标记点	-
$T_{\text{极限}}$	龙头前把手可用于调头的极限时间	s
$S_{\text{极限}}$	龙头前把手可用于调头的极限路程距离	m
S	调头阶段两段圆弧路径的连接点	-
E	龙头把手首次进入调头圈瞬间的位置标记点	-
E'	E 关于原点中心对称的标记点	-
R_{MS}	调头阶段第一个弧半径较大的圆弧 1 的圆弧半径	m
R_{NS}	调头阶段第二个弧半径较小的圆弧 2 的圆弧半径	m
θ_{MS}	调头圈中第一个圆弧所对的圆心角度数	○
θ_{NS}	调头圈中第二个圆弧所对的圆心角度数	○

8.2 模型建立

根据题意，龙头前把手在满足调头要求以及相切条件下，龙头的前把手必须要完成调头操作后再次返回到 E 点位置。则再次返回到 E 点位置后离开调头区域的运动过程与龙头的前把手进入调头区的运动过程相互对称，相当于模拟前一个运动过程的路径。

通过模型过程对称研究后，对于圆弧调头路径的最小值规划问题的建模假设如下：

1. 第四题的运动过程分解为三个子过程，并且进行优化改善：盘入、调头、盘出。
2. 假设盘入：盘入过程为龙头的前把手首次进入 E 点位置到即将进入调头阶段的前一

瞬间之间的时间段。

3. 假设调头：调头过程为龙头前把手在调头区以 S 型调头圆弧的运动过程。
4. 假设盘出：盘出过程为龙头的前把手从 E' 点位置再次到达 E 点过程的时间段。

由于同时需要满足调头的圆弧与盘入、盘出路线均相切，那么同时要满足极限情况下的相切条件：为简化模型研究，本模型假设龙头前把手盘入调头圈的瞬间速度与 E 处圆的切线方向一致。为了满足第四题新提出的螺距条件 (P=1.7m)，结合板凳的结构图，分析可知：

龙头在 E’ 点位置盘出过程中，整体龙头把手的极径向外偏转量 δr。

$$\Delta r = P - \left(P_0 - \frac{2 \times \text{width}}{2} \right) = 1.15\ \text{m} \quad (P_0\text{为原设定的螺距}) \tag{29}$$

由于盘入、盘出过程中，龙头把手整体呈现极径向外偏移的趋势，故：控制相同速度及相同极角运动偏移量一致的情况下，情况 B 在近似化成圆周的轨迹上运动时间 t_B 大于在相同条件下情况 A 的 t_A 运动时间，$t_A < t_B$。因此在寻找到最优化螺线运动路径后，可实现 t_A -> t_B (极限趋近)。

针对上述通过极限思想下提出的 t_A -> t_B 进一步分析，分析过程如下:

8.2.1 盘入、盘出过程

由上述分析可知，龙头把手整体呈现极径向外偏移的趋势，总体运动的路程大于光滑圆周。根据前面曲率的分析（借助前面图像 (惊悚图) 进行分析），可以得出以下结论：

- 随着运动过程的进行，不同时间对应龙头方向不同。
- 运动速度受曲率影响的程度不同且差异巨大。
- 曲率越小对速度的影响程度越小。

因此，在盘入和盘出过程中，若能选择理想的盘入方向，则把手受到曲率的影响程度达到最小化，更加近似光滑圆周轨道。

龙头前把手从 E’ 点运动再次到达 E 点位置过程的实际运动时间 tA, 盘出，可理想化为：

$t_{A,\text{盘出}} \to t_{B,\text{盘出}}$

$t_{A,\text{盘入}} \to t_{B,\text{盘入}}$

8.2.2 分析情况 B:

由严格圆周的理想情况 B，计算龙头前把手首次盘入到达 E 点的瞬间到完成三个子过程再次返回到该 E 点盘出瞬间的时间段 $T_{\text{情况 B}}$。

$$T_{\text{情况 B}} = \frac{S_{\text{总}}}{V}$$

$$= \frac{1}{2} \times \pi \times d \times 2 = \pi \times d$$

$$= 9\pi \approx 28.274333\,\text{s}, \quad v = 1\,\text{m/s} \tag{30}$$

8.2.3 分析情况 A 调头极限时间：

盘入、盘出情况：

龙头前把手在调头区的螺线运动范围，可近似为调头区的外围。故：初始半径 R_0= 4.5m。记：螺线从 E’点位置到 E 点位置逆时针运动的螺旋盘出路径弧长：L(m)

$$t_{A,\text{盘出}} = \frac{L}{V} \tag{31}$$

$$L = \int_{\theta_1}^{\theta_2} \sqrt{(R + b \cdot \theta)^2 + b^2}\, d\theta \tag{32}$$

通过数值积分操作，近似得到螺线弧长 $L \approx 15.495730\,\text{m}$。

$$t_{A,\text{盘出}} \approx 15.495730\,\text{s}$$

分析过程得到：

$$(t_{A,\text{盘入}} + t_{A,\text{调头}} + t_{A,\text{盘出}}) \rightarrow T_{\text{情况 B}} = 28.274333\,\text{s}$$

上述式子可理解为 $t_{A,\text{调头}}$ 的值无限趋近于圆周运动理想物体，龙头前把手可用于调头过程的极限时间为 $5.71002\,\text{s}$。由于匀速运动，同时可推断出极限路程的距离：

即：

$t_{\text{极限}} \approx 5.71002\,\text{s}$

$S_{\text{极限}} \approx 5.71002\,\text{m}$

8.2.4 分析情况 A 真实非线性规划与优化问题：

本题关注的是如何在给定的空间区域内（调头区）以及约束条件（弧相切）下设计与制定出板凳龙头把手 H 调头路径。从规划的角度，重点是如何找到满足所有约束条件的可行路径，并且关注路径长度的最小化。优化问题目标是找出角度调整的最佳参数；规划问题的目标是规划板凳龙头把手点 H 的行动路线，使得路径能够符合以下要求：

- 空间约束: 调头操作需要在规定空间内完成。

$$\sqrt{X_H^2 + Y_H^2} \leq R_{\text{调头}}$$

- 几何约束: 满足调头圆弧半径比例关系及调头圆弧与盘入、盘出螺线相切。

$$R_{MS} = 2 \times R_{NS},$$
$$X_E + X'_E = 0,$$
$$Y_E + Y'_E = 0$$

$$\det \begin{vmatrix} X_H - X_E & Y_H - Y_E \\ X_E - X_O & X_E - X_O \end{vmatrix} = 0$$

$$L_{\widehat{ME}} = L_{\widehat{NE'}}$$

- 运动约束: 龙头把手在行动时需要保证平稳的曲率变化，避免急转弯或过大半径。
- 极限约束: 龙头前把手调头操作的时间不大于理想状态下的极限时间。

$$t_{\text{调头}} \geq t_{\text{极限}}$$

- **非线性规划问题层次：**找到满足几何和运动约束的可行路径。
- **非线性优化问题层次：**在可行路径基础上，通过调整几何参数（如圆弧半径、曲线形状等）使得路径最优（最短、最平滑等）

8.2.4.1 非线性规划模型：几何结构的确定

规划问题主要是在满足一定几何和物理约束下设计路径。具体到这个问题，规划的关键在于设计出合理的路径形式和相应的几何条件。如图 8，

通过约束关系整理与几何关系的推理，得到目标函数的表示式如下：

$$f(\theta_{MS}) = \frac{\theta}{4} \sin\left(\frac{\theta}{2}\right) + \frac{\pi}{\sin\left(\frac{\theta}{2}\right)}$$

8.2.4.2 非线性优化模型：路径最短化

模型建立：

1、确定弧度制参数：此优化部分的目标是：最小化调头路径总长度所对应的调头圆弧所对的圆心角，结合上述的几何转化式，即：

目标函数为：

$$\min(R_{MS} \cdot \theta_{MS} + R_{NS} \cdot \theta_{NS}) \tag{33}$$

其中设定：

$$\text{函数 } 1 = R_{MS} \cdot \theta_{MS} + R_{NS} \cdot \theta_{NS}$$
$$\text{函数 } 2 = \frac{\theta}{4 \cdot \sin\left(\frac{\theta}{2}\right)} + \frac{\pi}{\sin\left(\frac{\theta}{2}\right)}$$

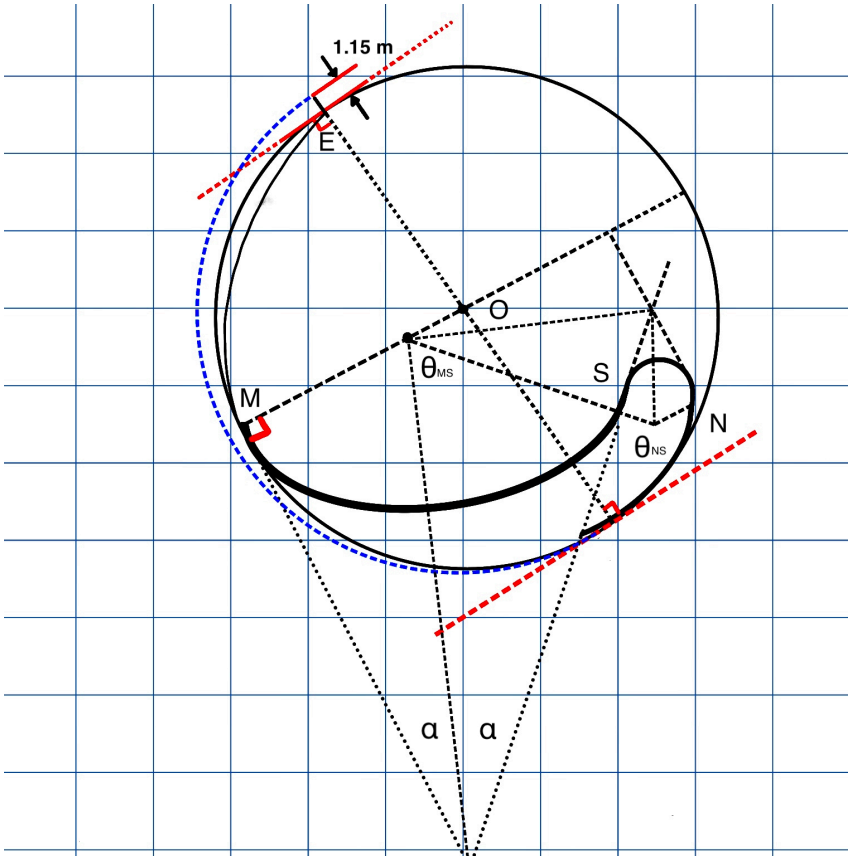


图 8　从 E 侧盘入调头空间满足约束情况的一种路径解示意图

2、确定半径参数：记：上一步分析过程中，确定某个弧度制下的直线记为 l’。第二段圆弧所对的圆心点分别记为：点 M’、点 N’。

由于调头的两段圆弧轨迹必须要求相切，因此两段圆弧的连接点必须在直线 l' 轨迹上。在弧度制参数确定后，第一个圆弧的偏转弧线轨迹的位置范围可以确定，接下来需要对第一个圆弧的偏转半径进行迭代分析：在调头圆弧所对圆心角 ($theta_MS$) 固定情况下，进行以下两种同步移动：在 1/4 圆周的半径线段及其延长线区域 (x 属于 (-4.5,4.5) 数值区间) 不断移动弧 (弧标记) 对应的圆心位置 M’，同时直线 l' 上的连接点 S 也同步移动，故轨迹可进行模拟且始终满足以下式子：

$$R_{MS} = L_{MM'} = L_{SM'} \tag{34}$$

$$R_{MS} = 2 \cdot R_{NS} \tag{35}$$

模型求解与分析：遗传算法优化初步分析：

$$\min_{\theta_{MS}} f(\theta_{MS}) = 5$$

$$
\begin{cases}
R_{MS} = 2 \cdot R_{NS} \\
\widehat{ME} = \widehat{NE'} \\
\det \begin{vmatrix} X_H - X_E & Y_H - Y_E \\ X_E - X_O & X_E - X_O \end{vmatrix} = 0 \\
X_E + X_{E'} = 0, \quad Y_E + Y_{E'} = 0 \\
t_{\text{极限}} \leq t_{\text{调头}} \\
\sqrt{X_h^2 + Y_h^2} \leq R_{\text{调头}}
\end{cases}
$$

使用数值优化方法，经圆弧旋转度数的取余操作后，计算出较长弧的 $\theta_M S$ 圆心角的角度为 51.324839 度. 优化算法的最优目标值的变化如下图所示。

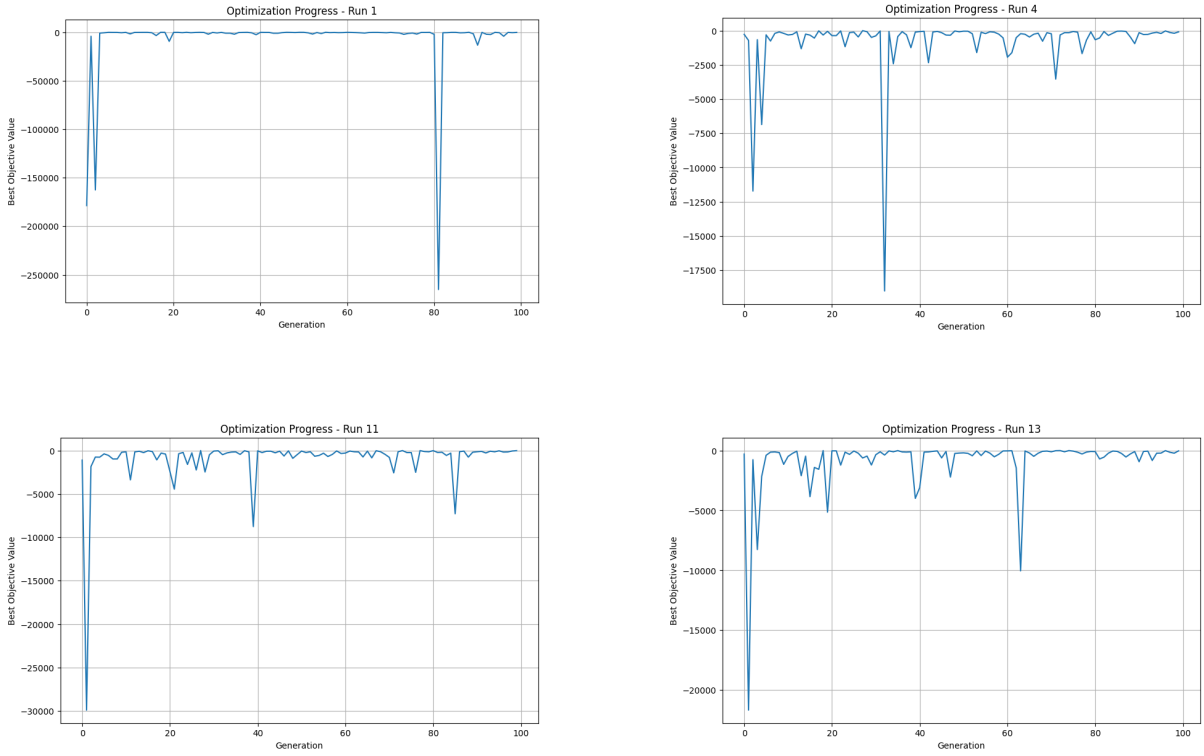


图 9　多次遗传算法迭代结果示意图

表 6　第 i 次遗传优化数据结果

第 i 次遗传优化过程	1	2	3	4
局部最优解的角度（○）	52.38378	46.61153999	67.70808	79.33112603
局部最优解的调头路径距离（m）	7.229822235	7.055369245	7.759362485	8.239529876

对于图 9 的可视化分析可知：优化的初期阶段 (Genertation<10)，图像上的目标函数值可能比较高，波动性较高，表明算法初期迭代阶段，个体距离最优解还较远；随着优化迭代次数的增加，关于距离优化的目标函数值应逐渐趋于正常数值的区间。这表明优化算法在逐步找到更优解。后期迭代的目标函数值趋于稳定，说明数值优化算法已经收敛到一个局部最优解的正常区间。

接下来通过角度的半径值的迭代，即：在 1/4 圆周的半径线段及其延长线区域 ($x \in (-4.5, 4.5)$) 不断移动第一段圆弧圆心 M’ 点，通过几何数量关系的约束，得到大致的调头运动轨迹。

8.3 求解结果

表 7　result4 中选取几个把手的位置

	-100 s	-50 s	0 s	50 s	100 s
龙头 x (m)	4.048473	5.637915	0.000000	-2.017800	-3.456281
龙头 y (m)	-1.977144	-4.770564	0.450000	-3.277145	-3.422154
第 1 节龙身 x (m)	6.227482	4.99752	-0.091724	-1.972440	-2.654325
第 1 节龙身 y (m)	-4.133217	-4.805342	4.472831	-3.344154	-3.978615
第 51 节龙身 x (m)	-2.489321	-5.112783	1.604561	2.137488	4.342908
第 51 节龙身 y (m)	6.549998	5.954620	5.988731	4.872921	-5.473092
第 101 节龙身 x (m)	-2.372703	5.281866	-4.801812	-6.271839	-1.045032
第 101 节龙身 y (m)	6.558664	6.919913	5.318122	-5.467682	2.563998
第 151 节龙身 x (m)	2.611342	-5.321011	5.480000	3.314253	-5.723619
第 151 节龙身 y (m)	7.944670	5.456660	-5.057410	-4.779885	-5.443788
第 201 节龙身 x (m)	-4.106631	4.834777	-5.533215	-4.003245	-2.867571
第 201 节龙身 y (m)	-7.613172	-4.394749	5.422151	5.357677	4.201877
龙尾（后）x (m)	-5.674828	-0.371954	6.794332	7.226556	5.398614
龙尾（后）y (m)	-2.211421	5.530439	-7.123457	6.189332	-1.482611

九、 问题五：龙头最大行进速度优化问题分析

9.1 模型建立与分析

假设龙头以恒定速度 V_{max}, 由于沿着设定的螺旋路径行进，我们可以使用阿基米德螺线方程来写出龙头的运动路径方程：

$$r(\theta) = 8.8 - \frac{1.7}{2\pi}\theta \tag{36}$$

由第一题求出整条龙各个把手的速度表格可视化得知，在盘入螺线的全过程中，龙头的速度永远比龙身和龙尾的速度略快，半径逐渐增大。由于龙头保持匀速，龙身和龙尾的速度将会逐渐减小，因为它们的路径曲率较大，导致速度较低。

已知“盘龙所需面积越小，行进速度越快”，求龙头的最大行进速度以提高观赏性。在前面第四题路径的基础上，我们需要进一步研究舞龙队每个把手在 S 形区域行进过程中的速度变化。已知龙身和龙尾的速度会受到路径曲率的影响，随着半径的增加，它们的速度会逐渐降低。盘入直径为 $R_{□□} = 9$m 的掉头空间时，舞龙队先进入半径较小圆弧，此时的半径小于 4.5m, 龙头线速度保持不变的情况下，舞龙队后面把手的速度会获得一个数值为正的加速度。

再进入半径较大的圆弧时，舞龙队后面把手的速度会获得一个数值为负的加速度。因此速度最大值（不确定是哪个把手获得全局最大值）会出现在半径较小的圆弧上。龙头沿着圆弧速度恒定不变的情况下，由于本题要求整条龙把手速度均不超过 2m/s，我们需要得知所有把手在前进的过程中速度最大的该把手的 i 值，求出表达式，临界值为该速度等于 2m/s。

9.2 求解结果

龙头把手的前进速度为 V_{max}, 我们可以使用粒子群算法进行优化，最后得出 V_{max} 的数值为 1.5m/s。

十、 模型的评价

10.1 模型的优点

- 全文的建模始终基于数学理性思维，利用平面几何知识、线性代数（旋转矩阵）、非线性方程、数值积分、迭代法、极限思想等数学理论以及相对论的时间延迟与相对距离的物理理论建立模型。
- 合理运用模拟退火，遗传算法等优化算法。
- 在问题一求解位置时，我们先基于时间延迟与相对距离的原理计算出位置，再使用数值积分（牛顿-柯斯特公式，五次曲线逼近法和辛普森法则）进行验证。

- 运行速度合理，资源利用率高。

10.2 模型的缺点

- 由于理想化模型导致误差存在，如问题四中最优路径只能找出局部最优解，与现实存在偏差。
- 没有考虑到板凳在三维空间的高度可能存在不相同的情况。

参考文献

[1] 刘柯, 陈秀政, 宋金城, 等. 阿基米德螺线在线性补偿应用中的原理误差分析[J]. 宇航计测技术, 2010, 30(06):62-66.

[2] COOPE I D. Curve fitting with nonlinear spiral splines: 63[R]. [S.l.]: Department of Mathematics, University of Canterbury, 1991.

[3] LăZUREANU C. Spirals on surfaces of revolution[J]. VisMath, 2014, 16:1-10.

[4] MASUREL C. A generalisation of sinusoidal spirals and ribaucour curves: Part iii[Z]. [S.l.: s.n.], 2015.

附录 A　文件列表

文件名	功能描述
运动动画.m	“板凳龙”在等距螺线运动模拟
result1 位置.py	问题一从初始时刻到 300s 为止，每秒整个舞龙队的位置求解程序
result1 速度.py	问题一从初始时刻到 300s 为止，每秒整个舞龙队的速度求解程序
result1.xlsx	问题一从初始时刻到 300s 为止，每秒整个舞龙队的位置与速度总表
顶点方程.py	问题二顶点运动轨迹绘画
result2 位置.py	问题二恰好未碰撞时刻整个舞龙队的位置求解程序
result2 速度.py	问题二恰好未碰撞时刻整个舞龙队的速度求解程序
result2.xlsx	问题二恰好未碰撞时刻整个舞龙队的位置与速度总表
问题三.py	问题三最小螺距优化程序
递归算法.py	解决递归过程
遗传优化算法.py	问题四路径优化代码

附录 B　代码

运动动画.m

```
% 初始化参数
initial_radius = 8.8;  % 初始半径，单位：米
pitch = 0.55;          % 螺距，单位：米
velocity = 1;          % 速度，单位：米/秒
duration = 300;        % 总时长，单位：秒
time_interval = 1;     % 时间间隔，单位：秒

% 计算每秒的角速度
omega = velocity / initial_radius;

% 时间向量
time = 0:time_interval:duration;

% 预分配位置矩阵
x = zeros(length(time), 1);
y = zeros(length(time), 1);
```

```
17
18  % 创建动画图形
19  figure;
20  hold on;
21  grid on;
22  axis equal;
23  xlabel('X 位置 (米)');
24  ylabel('Y 位置 (米)');
25  title('舞龙队沿螺旋线运动的动画');
26
27  % 创建动画
28  for i = 1:length(time)
29      t = time(i);
30      % 计算当前角度
31      theta = -omega * t;  % 负号表示顺时针方向
32      % 计算当前半径，随着时间减小
33      r = initial_radius - (pitch * abs(theta)) / (2 * pi);
34      % 计算当前位置
35      x(i) = r * cos(theta);
36      y(i) = r * sin(theta);
37
38      % 清除之前的图形
39      cla;
40      % 绘制当前时刻的位置
41      plot(x(1:i), y(1:i), 'b-o', 'LineWidth', 1.5, 'MarkerSize'
        , 4);
42      xlim([min(x)-1 max(x)+1]);
43      ylim([min(y)-1 max(y)+1]);
44      pause(0.1);  % 控制动画的播放速度
45  end
46
47  % 保存动画为视频文件
48  vidObj = VideoWriter('dragon_animation', 'MPEG-4');
49  open(vidObj);
50
```

```
for i = 1:length(time)
    t = time(i);
    % 计算当前角度
    theta = -omega * t;  % 负号表示顺时针方向
    % 计算当前半径，随着时间减小
    r = initial_radius - (pitch * abs(theta)) / (2 * pi);
    % 计算当前位置
    x(i) = r * cos(theta);
    y(i) = r * sin(theta);

    % 清除之前的图形
    cla;
    % 绘制当前时刻的位置
    plot(x(1:i), y(1:i), 'b-o', 'LineWidth', 1.5, 'MarkerSize', 4);
    xlim([min(x)-1 max(x)+1]);
    ylim([min(y)-1 max(y)+1]);
    frame = getframe(gcf);
    writeVideo(vidObj, frame);
    pause(0.1);  % 控制动画的播放速度
end
```

result1 位置.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 定义螺旋线参数
initial_radius = 8.8  # 初始半径，单位为米
pitch = 0.55          # 螺距，单位为米
velocity = 1          # 龙头速度，单位为米每秒
duration = 300        # 持续时间，单位为秒
time_interval = 1     # 时间间隔，单位为秒
dragon_head_length = 2.86  # 龙头到龙身第一个点的距离，单位为米
```

```
dragon_body_length = 1.65  # 龙身两点之间的距离，单位为米

# 时间序列
time = np.arange(0, duration + time_interval, time_interval)

# 点的数量
num_points = 224

# 初始化位置矩阵和角度矩阵
x = np.zeros((num_points, len(time)))
y = np.zeros((num_points, len(time)))

# 计算每个时刻的龙头位置
for i, t in enumerate(time):
    # 计算龙头（第一个点）的角度
    theta_head = velocity * t / initial_radius  # 角度
    # 计算龙头的半径
    r_head = initial_radius - (pitch * theta_head) / (2 * np.pi)

    # 计算龙头的坐标
    x[0, i] = r_head * np.cos(theta_head)
    y[0, i] = -r_head * np.sin(theta_head)

    # 对于龙身的每个点，逐个计算其位置
    for n in range(1, num_points):
        # 确定距离龙头的总距离
        if n == 1:
            total_distance = dragon_head_length
        else:
            total_distance = dragon_head_length + (n - 1) * dragon_body_length

        # 通过距离计算当前点相对于龙头的时间延迟
        time_delay = total_distance / velocity
```

```
45
46          # 调整时间，使得每个点基于其距离龙头的时间延迟
47          t_adjusted = max(0, t - time_delay)
48
49          # 计算当前点的角度和半径，保持与龙头相同的螺线轨迹
50          theta = velocity * t_adjusted / initial_radius
51          r = initial_radius - (pitch * theta) / (2 * np.pi)
52
53          # 计算点的坐标
54          x[n, i] = r * np.cos(theta)
55          y[n, i] = -r * np.sin(theta)
56
57  # 创建一个字典来存储 DataFrame 的列
58  data = {'时间 (秒)': time}
59  for n in range(num_points):
60      data[f'点 {n+1} x (米)'] = x[n, :]
61      data[f'点 {n+1} y (米)'] = y[n, :]
62
63  # 创建 DataFrame
64  df = pd.DataFrame(data)
65
66  # 将结果保存到 Excel 文件
67  with pd.ExcelWriter('result1.xlsx', engine='openpyxl') as
        writer:
68      df.to_excel(writer, index=False, sheet_name='Positions')
69
70  print("位置已保存到 'result1.xlsx'")
71
72
73
74
75  # 绘制图像
76  for n in range(num_points):
77      plt.plot(x[n, :], y[n, :], label=f'点 {n+1}', marker='o',
        markersize=2, alpha=0.7)
```

```
78
79  # 设置坐标轴和标题
80  plt.xlabel('X (m)')
81  plt.ylabel('Y (m)')
82  plt.title('Positions of Points Along the Spiral Path')
83
84  # 调整图例位置和字体大小
85  plt.legend(loc='upper left', bbox_to_anchor=(1, 1), fontsize='
        xx-small', ncol=10)
86
87  # 调整图像边距
88  plt.tight_layout(rect=[0, 0, 0.8, 1])
89
90  # 显示图像
91  plt.grid(True)
92  plt.gca().set_aspect('equal', adjustable='box')
93  plt.show()
```

result1 速度.py

```
1  import numpy as np
2  import pandas as pd
3
4  # 使用 genfromtxt 读取 CSV 文件并处理空值
5  x = np.genfromtxt('速度/x.csv', delimiter=',', skip_header=1,
       filling_values=np.nan)
6  y = np.genfromtxt('速度/y.csv', delimiter=',', skip_header=1,
       filling_values=np.nan)
7
8  # 打印数据以确保正确读取
9  print(x)
10 print(y)
11
12 print("\n")
13 print(x[0][1])
14 print(x[1][1])
```

```
print(y[0][1])
print(y[1][1])

V0 = 1
R = 8.8
b = 0.0875

speed_list = []

# 从正常时间非起点开始计算 (t = 1开始计算)
for row in range(1, 300):  # 由于后面有一个 + 1 的索引值，所以不能取得 301， 只能取到 300
    new_V = V0

    single_time_speed = [new_V]

    for col in range(0, 223):  # 由于后面有一个 + 1 的索引值，所以不能取得 224， 只能取到 223
        point_x = x[row][col]
        point_y = y[row][col]

        x1 = x[row][col]
        y1 = y[row][col]

        x2 = x[row + 1][col + 1]
        y2 = y[row + 1][col + 1]

        theta1 = np.arctan2(y1, x1)
        theta2 = np.arctan2(y2, x2)

        new_V *= np.float64(np.sqrt(
            np.float64((R + b * theta2) ** 2 + b ** 2) / \
            np.float64((R + b * theta1) ** 2 + b ** 2)
        ))

```

```
        single_time_speed.append(new_V)

    speed_list.append(single_time_speed)

speed_list.append(speed_list[10])

print("\n")
print(np.array(speed_list).shape)
print(np.array(speed_list))

# 将速度数据写入 CSV 文件
df = pd.DataFrame(speed_list)
df.to_csv('result12.csv', index=False)

# 读取原始 CSV 文件
df = pd.read_csv('result12.csv')

# 确保新行长度与 DataFrame 列数一致
num_columns = df.shape[1]
new_row = [1] + [0] * (num_columns - 1)

# 将新行添加到 DataFrame 的第一行
df.loc[-1] = new_row  # 添加新行到 DataFrame
df.index = df.index + 1  # 调整索引
df = df.sort_index()  # 重新排序 DataFrame 按照索引

# 将更新后的 DataFrame 写入到新的 CSV 文件
df.to_csv('result12.csv', index=False,float_format='%.6f')
print((np.array(speed_list)).shape)
```

result2 位置.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import fsolve
```

```
5
6  # 螺旋线参数
7  initial_radius = 8.8
8  pitch = 1.0
9  velocity = 1  # 速度 1m/s
10 dragon_head_length = 2.86
11 dragon_body_length = 1.65
12 distance_threshold = 0.15  # 垂直距离阈值
13
14 # 固定距离点参数
15 d = np.sqrt(distance_threshold**2 + 0.275**2)  # 固定距离
16 angle_a = np.arctan(-15 / 27.5)  # 固定角度
17
18 # 定义螺旋线位置函数，计算龙头或身体节段的位置
19 def calculate_position(t, n):
20     D_i = dragon_head_length + (n - 1) * dragon_body_length
21     time_delay = D_i / velocity
22     t_adjusted = t - time_delay
23     theta = velocity * t_adjusted / initial_radius
24     r = initial_radius - (pitch * theta) / (2 * np.pi)
25     x = r * np.cos(theta)
26     y = -r * np.sin(theta)
27     return x, y
28
29 # 计算固定点的位置
30 def fixed_point_position(t, i, n):
31     if i <= n + 2:
32         raise ValueError("i must be greater than n + 2")
33
34     theta_head = velocity * t / initial_radius
35     r_head = initial_radius - (pitch * theta_head) / (2 * np.
   pi)
36     x_head = r_head * np.cos(theta_head)
37     y_head = -r_head * np.sin(theta_head)
38
```

```
    # 计算当前点的位置
    D_i = dragon_head_length + (i - 1) * dragon_body_length
    time_delay = D_i / velocity
    t_adjusted = t - time_delay
    theta = velocity * t_adjusted / initial_radius
    r = initial_radius - (pitch * theta) / (2 * np.pi)
    x = r * np.cos(theta)
    y = -r * np.sin(theta)

    x_relative = d * np.cos(angle_a - theta)
    y_relative = d * np.sin(angle_a - theta)

    x_point = x_head + x_relative
    y_point = y_head + y_relative

    return x_point, y_point

# 定义垂直距离函数，计算龙头与第i节段之间的垂直距离
def vertical_distance(t, i, n):
    if i <= n + 2:
        raise ValueError("i must be greater than n + 2")
    x_0, y_0 = fixed_point_position(t, i, n)
    x_i, y_i = calculate_position(t, i)
    x_i1, y_i1 = calculate_position(t, i + 1)

    numerator = abs((y_i1 - y_i) * x_0 - (x_i1 - x_i) * y_0 +
   x_i1 * y_i - y_i1 * x_i)
    denominator = np.sqrt((y_i1 - y_i)**2 + (x_i1 - x_i)**2)
    return numerator / denominator

# 定义求解方程，找到满足垂直距离条件的时间
def equation(t, i, n):
    return vertical_distance(t, i, n) - distance_threshold

# 搜索满足条件的唯一 i 和 t
```

```
def find_unique_solution():
    initial_guess = 10  # 初始猜测值
    for n in range(1, 224):  # n 从 1 到 223
        for i in range(n + 3, 224):  # 确保 i 大于 n + 2
            try:
                t_solution = fsolve(equation, initial_guess, args=(i, n))
                distance = vertical_distance(t_solution[0], i, n)

                if abs(distance - distance_threshold) < 0.025:  # 判断是否满足阈值
                    return t_solution[0], i, n
            except ValueError:
                continue  # 如果出现 ValueError，则继续寻找其他 i 和 n 的组合

    return None, None, None

# 查找碰撞时间点
t_collision, i_collision, n_value = find_unique_solution()

if t_collision is not None:
    print(f"唯一满足垂直距离为 {distance_threshold}m 的时间点为：{t_collision:.6f} 秒")
    print(f"此时 i = {i_collision} 和 i+1 = {i_collision + 1}")

    # 计算碰撞时间下所有点的位置
    num_points = 224
    x_positions = np.zeros(num_points)
    y_positions = np.zeros(num_points)

    for n in range(num_points):
        x, y = calculate_position(t_collision, n)
```

```
        x_positions[n] = x
        y_positions[n] = y

    # 为 Excel 文件生成点标签
    labels = ["龙头"] + [f"第{i}节龙身" for i in range(1, 222)] + ["龙尾"] + ["龙尾（后）"]

    # 保存结果到 Excel 文件
    results_collision = [{"    ": labels[n], "横坐标x/(m)": x_positions[n], "纵坐标y/(m)": y_positions[n]} for n in range(num_points)]
    df_collision = pd.DataFrame(results_collision)
    df_collision.to_excel("result2.xlsx", index=False)

else:
    print("没有找到满足条件的时间点。")
```

result2 速度.py

```
import numpy as np
import pandas as pd

# 使用 genfromtxt 读取 CSV 文件并处理空值
x = np.genfromtxt('副本result2.csv', delimiter=',', skip_header=1, filling_values=np.nan)
x = np.genfromtxt('副本result2.csv', delimiter=',', skip_header=1, filling_values=np.nan)
x = list(x[:, 1])

y = np.genfromtxt('副本result2.csv', delimiter=',', skip_header=1, filling_values=np.nan)
y = list(y[:, 2])


V0 = 1
R = 8.8
```

```
b = 0.0875

speed_list = []
speed_list.append(V0)

new_V = V0

# # 从正常时间非起点开始计算 (t = 1开始计算)
for row in range(0, 223):  #
    x1 = x[row]
    y1 = y[row]

    x2 = x[row + 1]
    y2 = y[row + 1]

    theta1 = np.arctan2(y1, x1)
    theta2 = np.arctan2(y2, x2)

    new_V *= np.float64(np.sqrt(
        np.float64((R + b * theta2) ** 2 + b ** 2) / \
        np.float64((R + b * theta1) ** 2 + b ** 2)
        ))

    speed_list.append(new_V)

print("\n")
print(np.array(speed_list).shape)
print(np.array(speed_list))

# 将速度数据写入 CSV 文件
df = pd.DataFrame(speed_list)
df.to_csv('result22.csv', index=False)

# 读取原始 CSV 文件
df = pd.read_csv('result22.csv')
```

```
num_columns = df.shape[1]
new_row = [1] + [0] * (num_columns - 1)

df.loc[-1] = new_row  # 添加新行到 DataFrame
df.index = df.index + 1  # 调整索引
df = df.sort_index()  # 重新排序 DataFrame 按照索引

df.to_csv('result22.csv', index=False,float_format='%.6f')
print((np.array(speed_list)).shape)
```

顶点方程.py

```
import numpy as np
import matplotlib.pyplot as plt

# 常量
initial_radius = 8.8
pitch = 0.55
velocity = 1
duration = 800
time_interval = 1
d = np.sqrt(0.15**2 + 0.275**2)  # 固定距离
angle_a = np.arctan(-15 / 27.5)  # 固定角度

time = np.arange(0, duration + time_interval, time_interval)

# 计算每个时刻点的位置
x_point = np.zeros(len(time))
y_point = np.zeros(len(time))

for i, t in enumerate(time):
    theta_head = velocity * t / initial_radius
    r_head = initial_radius - (pitch * theta_head) / (2 * np.
   pi)

    # 计算龙头位置
```

```
    x_head = r_head * np.cos(theta_head)
    y_head = -r_head * np.sin(theta_head)

    # 计算第一个点位置
    x_first_point = (initial_radius - (pitch * (velocity * t / initial_radius))) * np.cos(theta_head)
    y_first_point = -(initial_radius - (pitch * (velocity * t / initial_radius))) * np.sin(theta_head)

    # 计算当前点的位置
    x_relative = d * np.cos(angle_a - theta_head)
    y_relative = d * np.sin(angle_a - theta_head)

    # 点相对于绝对坐标系的坐标
    x_point[i] = x_head + x_relative
    y_point[i] = y_head + y_relative

# 绘制点的路径
plt.plot(x_point, y_point, label='Fixed Distance Point', marker='o', markersize=2, alpha=0.7)
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
plt.title('Position of Fixed Distance Point Along the Spiral Path (The first point)')
plt.grid(True)
plt.gca().set_aspect('equal', adjustable='box')
plt.legend()
plt.show()
```

问题三.py

```
import numpy as np
from pyswarm import pso
from scipy.optimize import fsolve

# 螺旋线参数
```

```
initial_radius = 8.8
velocity = 1
dragon_head_length = 2.86
dragon_body_length = 1.65
distance_threshold = 0.15  # 垂直距离阈值

# 固定距离点参数
d = np.sqrt(distance_threshold**2 + 0.275**2)  # 固定距离
angle_a = np.arctan(-15 / 27.5)  # 固定角度

# 全局 pitch 变量
pitch = 1.0  # 初始值

# 更新 pitch 的函数
def update_pitch(new_pitch):
    global pitch
    pitch = new_pitch[0]

# 定义螺旋线位置函数，计算龙头或身体节段的位置
def calculate_position(t, n):
    D_i = dragon_head_length + (n - 1) * dragon_body_length
    time_delay = D_i / velocity
    t_adjusted = t - time_delay
    theta = velocity * t_adjusted / initial_radius
    r = initial_radius - (pitch * theta) / (2 * np.pi)
    x = r * np.cos(theta)
    y = -r * np.sin(theta)
    return x, y

# 计算固定点的位置
def fixed_point_position(t, i, n):
    if i <= n + 2:
        raise ValueError("i must be greater than n + 2")

    theta_head = velocity * t / initial_radius
```

```
    r_head = initial_radius - (pitch * theta_head) / (2 * np.pi)
    x_head = r_head * np.cos(theta_head)
    y_head = -r_head * np.sin(theta_head)

    # 计算当前点的位置
    D_i = dragon_head_length + (i - 1) * dragon_body_length
    time_delay = D_i / velocity
    t_adjusted = t - time_delay
    theta = velocity * t_adjusted / initial_radius
    r = initial_radius - (pitch * theta) / (2 * np.pi)
    x = r * np.cos(theta)
    y = -r * np.sin(theta)

    x_relative = d * np.cos(angle_a - theta)
    y_relative = d * np.sin(angle_a - theta)

    x_point = x_head + x_relative
    y_point = y_head + y_relative

    return x_point, y_point

# 定义垂直距离函数，计算龙头与第i节段之间的垂直距离
def vertical_distance(t, i, n):
    if i <= n + 2:
        raise ValueError("i must be greater than n + 2")
    x_0, y_0 = fixed_point_position(t, i, n)
    x_i, y_i = calculate_position(t, i)
    x_i1, y_i1 = calculate_position(t, i + 1)

    numerator = abs((y_i1 - y_i) * x_0 - (x_i1 - x_i) * y_0 + x_i1 * y_i - y_i1 * x_i)
    denominator = np.sqrt((y_i1 - y_i)**2 + (x_i1 - x_i)**2)
    return numerator / denominator

```

```
# 定义求解方程，找到满足垂直距离条件的时间
def equation(t, i, n):
    return vertical_distance(t, i, n) - distance_threshold

# 搜索满足条件的唯一 i 和 t
def find_unique_solution():
    initial_guess = 10  # 初始猜测值
    for n in range(1, 224):  # n 从 1 到 223
        for i in range(n + 3, 224):  # 确保 i 大于 n + 2
            try:
                t_solution = fsolve(equation, initial_guess, args=(i, n))
                distance = vertical_distance(t_solution[0], i, n)

                if abs(distance - distance_threshold) < 0.025:  # 判断是否满足阈值
                    return t_solution[0], i, n
            except ValueError:
                continue  # 如果出现 ValueError，则继续寻找其他 i 和 n 的组合

    return None, None, None

# 查找碰撞时间点
def find_time_for_radius_constraint():
    def radius_constraint(t):
        x_head, y_head = calculate_position(t, 0)
        return x_head**2 + y_head**2 - 4.5**2

    t_solution = fsolve(radius_constraint, 10)
    return t_solution[0]

# 定义目标函数来计算两个时间点之间的差异
def objective_function(pitch):
```

```
    update_pitch([pitch])  # 确保 pitch 作为列表传递给 update_pitch
    t_collision, i_collision, n_value = find_unique_solution()
    if t_collision is None:
        return np.inf  # 如果没有找到合适的时间点，返回无穷大

    t_radius_constraint = find_time_for_radius_constraint()
    return abs(t_collision - t_radius_constraint)

# 设置粒子群优化的范围
lb = [0.1]  # lower bound
ub = [1.7]  # upper bound

result = pso(objective_function, lb, ub)

optimal_pitch = result[0][0]
print(f"优化后的pitch值为: {optimal_pitch:.6f}")

update_pitch([optimal_pitch])
t_collision, i_collision, n_value = find_unique_solution()
t_radius_constraint = find_time_for_radius_constraint()

print(f"优化后，唯一满足垂直距离为 {distance_threshold}m 的时间点为: {t_collision:.6f} 秒")
print(f"优化后，当龙头的x坐标的平方 + y坐标的平方 = {4.5**2} 时的时间点为: {t_radius_constraint:.6f} 秒")
```

递归算法.py

```
import numpy as np
import pandas as pd

# 使用 genfromtxt 读取 CSV 文件并处理空值
x = np.genfromtxt('速度/x.csv', delimiter=',', skip_header=1, filling_values=np.nan)
y = np.genfromtxt('速度/y.csv', delimiter=',', skip_header=1,
```

```
   filling_values=np.nan)

V0 = 1
R = 8.8
b = 0.0875

# 递归函数
def calculate_speed(row, col, current_V, max_row=300, max_col
   =223):
    if row >= max_row or col >= max_col:
        return []

    new_V = current_V
    single_time_speed = [new_V]

    for c in range(1, max_col):
        point_x = x[row][c]
        point_y = y[row][c]

        x1 = x[row][c]
        y1 = y[row][c]

        x2 = x[row + 1][c + 1]
        y2 = y[row + 1][c + 1]

        theta1 = np.arctan2(y1, x1)
        theta2 = np.arctan2(y2, x2)

        new_V *= np.float64(np.sqrt(
            np.float64((R + b * theta2) ** 2 + b ** 2) / \
            np.float64((R + b * theta1) ** 2 + b ** 2)
        ))
        single_time_speed.append(new_V)

    # 递归计算下一行
```

```
    speed_list = [single_time_speed] + calculate_speed(row + 1, col, new_V, max_row, max_col)
    return speed_list

# 初始化递归
speed_list = calculate_speed(1, 1, V0)

# 将速度数据写入 CSV 文件
df = pd.DataFrame(speed_list)
df.to_csv('output递归.csv', index=False)

# 读取原始 CSV 文件
df = pd.read_csv('output递归.csv')

# 确保新行长度与 DataFrame 列数一致
num_columns = df.shape[1]
new_row = [1] + [0] * (num_columns - 1)
df.loc[-1] = new_row  # 添加新行到 DataFrame
df.index = df.index + 1  # 调整索引
df = df.sort_index()  # 重新排序 DataFrame 按照索引

df.to_csv('output递归.csv', index=False, float_format='%.6f')
```

遗传优化算法.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# 目标函数
def objective_function(theta):
    return (theta / 4 * np.sin(theta / 2))  + np.pi / np.sin(theta / 2)

# 遗传算法参数
pop_size = 1000
```

```
num_generations = 100
mutation_rate = 0.02
tournament_size = 5
min_theta = 0.1  # 避免除以零
max_theta = 10.0
num_runs = 100  # 运行的次数

# 初始化种群
def initialize_population(pop_size, min_theta, max_theta):
    return np.random.uniform(min_theta, max_theta, pop_size)

# 计算适应度
def evaluate_population(population):
    return np.array([objective_function(theta) for theta in
   population])

# 选择操作
def select_population(population, fitness):
    selected_indices = np.argsort(fitness)[:pop_size]
    return population[selected_indices]

# 交叉操作
def crossover(parent1, parent2):
    return (parent1 + parent2) / 2

# 变异操作
def mutate(theta, mutation_rate, min_theta, max_theta):
    if np.random.rand() < mutation_rate:
        return np.random.uniform(min_theta, max_theta)
    return theta

# 遗传算法主函数
def genetic_algorithm():
    population = initialize_population(pop_size, min_theta,
   max_theta)
```

```
    best_solutions = []

    for generation in range(num_generations):
        fitness = evaluate_population(population)
        best_solutions.append(np.min(fitness))

        # 选择操作
        selected_population = select_population(population,
    fitness)

        # 生成新种群
        new_population = []
        while len(new_population) < pop_size:
            parents = np.random.choice(selected_population, 2)
            child = crossover(parents[0], parents[1])
            child = mutate(child, mutation_rate, min_theta,
    max_theta)
            new_population.append(child)

        population = np.array(new_population)

    # 获取最佳解
    best_fitness_index = np.argmin(fitness)
    best_theta = population[best_fitness_index]
    best_value = objective_function(best_theta)

    return best_theta, best_value, best_solutions

# 执行多次遗传算法
results = []
for run in range(num_runs):
    best_theta, best_value, best_solutions = genetic_algorithm
    ()

    # 调试信息
```

```
    print(f"Run {run + 1} - Best value: {best_value} (Range: [7, 10])")

    if 7 <= best_value <= 10:
        results.append((best_theta, best_value, best_solutions))

# 打印符合条件的结果
if results:
    for i, (best_theta, best_value, best_solutions) in enumerate(results):
        print(f"Run {i + 1}:")
        print(f"  Best theta: {best_theta}")
        print(f"  Best value: {best_value}")

        # 绘制优化过程图像
        plt.figure(figsize=(10, 6))
        plt.plot(best_solutions)
        plt.xlabel('Generation')
        plt.ylabel('Best Objective Value')
        plt.title(f'Optimization Progress - Run {i + 1}')
        plt.grid(True)
        plt.show()
else:
    print("No results found within the range [7, 10].")
```